

NAME: _____

POINTS: _____ / 10

1. What will the output of this code be?

```
const int SIZE = 4;
int results[SIZE] = { 54, 44, 34, 22 };
int *resultsPtr = nullptr;
resultsPtr = results + 2;
cout << *resultsPtr << endl;
```

Output:

2. Assume `ptr` is a pointer to an `int` and holds the address 12000. On a system with 4-byte integers, what address will be in `ptr` after the following statement?

```
ptr += 10;
```

Answer:

3. Could you show this with code?

```
int x;
int *ptr = &x;                                     // show old address

ptr += 10;                                           // show new address
```

4. Assume `pint` is a pointer-to-int variable. Is each of the following statements valid or invalid? If any is invalid, why?

```
A) pint++;
B) --pint;
C) pint /= 2;
D) pint *= 4;
E) pint += x; // x is an int
```

5. In the following code, is the pointer variable definition legal or illegal?

```
int seats[5] = {2,4,5,7,6};
int *seats = seats;
```

Answer:

NAME: _____

POINTS: _____ / 10

6. In the following code, is the assignment statement in the 2nd line legal or illegal?

```
double dollars;  
int dollarsPtr = &dollars;
```

Answer:

7. Is each of the following pointer definitions and initializations valid or invalid? If any is invalid, why?

```
A)  int ivar;  
    int *iptr = &ivar;  
  
B)  int ivar, *iptr = &ivar;  
  
C)  float fvar;  
    int *iptr = &fvar;  
  
D)  int nums[50], *iptr = nums;  
  
E)  int *iptr = &ivar;  
    int ivar;
```

8. Will this function compile?

```
void getNumber(int *input) {  
    cout << "Enter an integer number: ";  
    cin >> input;  
}  
int main(){int num; getNumber(num);}
```

Answer:

9. Rewrite this pointer-based function to use a reference parameter instead. Output is 10.

```
void increment(int *ptr) { (*ptr)++; }  
  
int main() {  
    int x = 9;  
    increment(&x);  
    cout << x;  
}
```

NAME: _____

POINTS: _____ / 10

10. The function `copyInts` copies an array `src` into an array `dst`. Replace the array parameters by pointers, and replace all indexing with pointer arithmetic.

```
// Copy n ints from src to dst using pointer parameters.
void copyInts(int dst[], const int src[] int n) {
    for (int i=0; i<n; i++)
    {
        // assign from current src element to current dst element
        dst[i] = src[i];
    }
}
```

Solution: